To pay more attention to gain better result

ASSEMLY LANGUAGE 02

Objectives

After studying this chapter, you should be able to:

- Perform arithmetic operations
- Access variable's address
- Draw the memory map of a program
- Understand the way procedures work.
- Use a loop operation

Contents

- 1- Arithmetic operations
- 2- Access variable's address and memory map
- 3- Procedure with pointer parameters
- 4- Use Loops

1- Arithmetic Operations

EX06_Adding.asm

Write a MASM program that will accept 2 integers, then sum of them will be print out.

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

K:\GiangDay\FU\CEA\ASM-Examples>ex06_sum
Enter number 1 : 12
Enter number 2 : -890
Sum of them:-878
K:\GiangDay\FU\CEA\ASM-Examples>ex06_sum
Enter number 1 : -90
Enter number 2 : -10
Sum of them:-100
K:\GiangDay\FU\CEA\ASM-Examples>ex06_sum
Enter number 1 : 34
Enter number 2 : -34
Sum of them:0
K:\GiangDay\FU\CEA\ASM-Examples>_
```

EX06_Adding.asm - Source code

```
; EX06_Adding.asm   Accept 2 integers, sum of them will be printed out
include \masm32\include\masm32rt.inc
sum PROTO :DWORD, :DWORD        ;  prototype a method + 2 parameters
.code
start:                          ; The CODE entry point of the program
    call main                   ; branch to the "main" procedure
    exit
; «««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««««
main proc
    LOCAL var1:DWORD            ; 2 DWORD integral variables
    LOCAL var2:DWORD            ;
    LOCAL result:DWORD          ; Result of operation
  ; Input 2 integers
    mov var1, sval(input("Enter number 1 : "))
    mov var2, sval(input("Enter number 2 : "))
  ; Invoke the procedure SUM to compute their sum, result in EAX
    push eax                    ; store EAX to STACK
    invoke sum, var1 , var2
    mov result, eax             ; result = EAX
    pop eax                     ; restore EAX from STACK
```

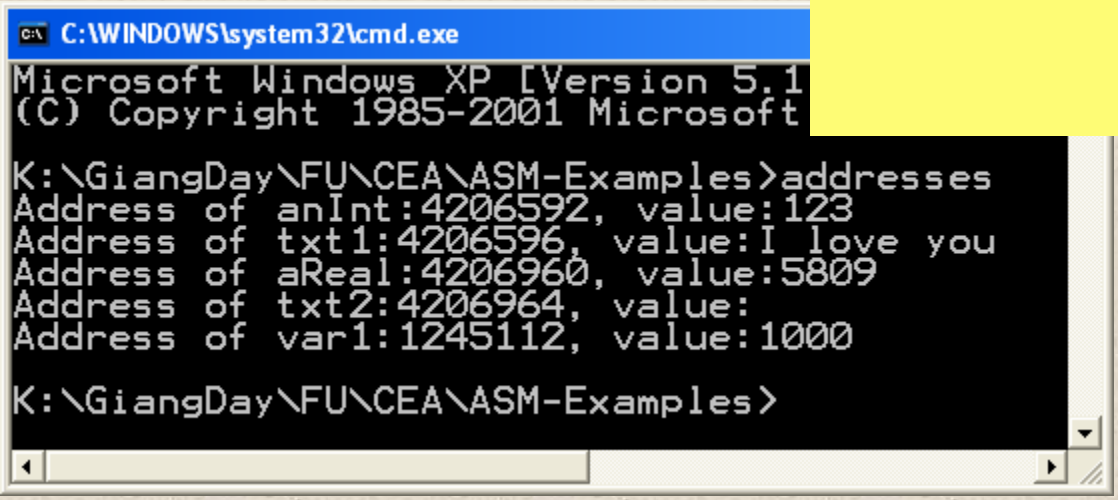
EX06_Adding.asm - Source code

```
 ; Print the result
    print chr$("Sum of them:")
    print str$(result)

    ret
main endp
; «««««««««««««««««««««««««««««««««««««««««
sum proc v1: DWORD, v2:DWORD
   mov eax, v1          ; eax= v1
   add eax, v2          ; eax = eax + v2 -> Result in eax
   ret
sum endp

end start
```

2- Access Variable's Address

- The following program depicts how to get addresses of variables:
  - The operator OFFSET for global variables
  - The instruction LEA for local variables
  - Draw memory map of a program

Addresses.asm- Access Variable's Address



Data segment (global variables)

(txt2)4206964

(aReal)4206960

(txt1)4206596

(anInt)4206592

(var1)1245112

Stack segment (local variables)

5809

I love you

123

1000

Addresses.asm- Access Variable's Address

```
; Addresses.asm
; Draw memory of a program

include \masm32\include\masm32rt.inc

.data              ; initialized data
  anInt DD 123
  txt1 db "I love you", 0

.data?              ; Un-initialized data
  aReal DD ?
  txt2  db 128 dup (?)

.code
start:
    call main
    exit
; <<<
```

Addresses.asm- Access Variable's Address

```
; Addresses.asm
; Draw memory of a program

include \masm32\include\masm32rt.inc

.data           ; Initialized data
  anInt DD 123
  txt1 db "I love you", 0

.data?          ; Un-initialized data
  aReal DD ?
  txt2  db 128 dup (?)

.code
start:
    call main
    exit
; <<<
```

Addresses.asm- Source code

```
; Addresses.asm
; Draw memory of a program

include \masm32\include\masm32rt.inc
 .data          ; initialized data
  anInt DD 123
  txt1 db "I love you", 0
 .data?          ; Un-initialized data
  aReal DD ?
  txt2  db 128 dup (?)
 .code
start:                          ; The CODE entry point to the program
    call main                   ; branch to the "main" procedure
    exit
; «««««
main proc
    LOCAL var1: DWORD

 ; Access address and value of anInt
    print chr$("Address of anInt:")
    mov eax, OFFSET anInt ; Operator OFFSET will get address of a global var
    print str$(eax)
    print chr$(", value:")
    print str$(anInt)
    print chr$(13,10)
 ; Access address and value of txt1
    print chr$("Address of txt1:")
    mov eax, OFFSET txt1
    print str$(eax)
    print chr$(", value:")
    print OFFSET txt1
    print chr$(13,10)
```

```
 ; Access address and value of aReal
    print chr$("Address of aReal:")
    mov eax, OFFSET aReal
    print str$(eax)
    print chr$(", value:")
    mov aReal, 5809
    print str$(aReal)
    print chr$(13,10)
  ; Access address and value of txt2
    print chr$("Address of txt2:")
    mov eax, OFFSET txt2
    print str$(eax)
    print chr$(", value:")
    print OFFSET txt2
    print chr$(13,10)
  ; Access address and value of local var var1
    mov var1, 1000
    print chr$("Address of var1:")
    lea eax, var1  ; LEA get local variable address
    print str$(eax)
    print chr$(", value:")
    print str$(var1)
    print chr$(13,10)
    ret
main endp
; «««««««««««««««««««««««««
end start
```

+ Swap1.asm

- This program depicts passing values to arguments when calling a procedure
- Program that will accept 2 integers, swap them then print out results.
- This version can not perform requirements successfully because the procedure **swap1** accesses variables directly

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

K:\GiangDay\FU\CEA\ASM-Examples>swap1
Enter number 1 : 123
Enter number 2 : 456
var1, address:1245112, value:123
var2, address:1245104, value:456
Argument 1, address:1245084, value:123
Argument 2, address:1245088, value:456
After swapping:123, 456
K:\GiangDay\FU\CEA\ASM-Examples>_
```

Comment:
This program can not swap 2 values

Swap1.asm – Source code

```
; Swap1.asm   Accept 2 integers, swap them, print results
include \masm32\include\masm32rt.inc
swap1 PROTO :DWORD, :DWORD ;  prototype the procedure swap1

.code
start:
    call main
    exit
; <<<
```

Swap1.asm – Memory Map when the procedure Swap1 is called

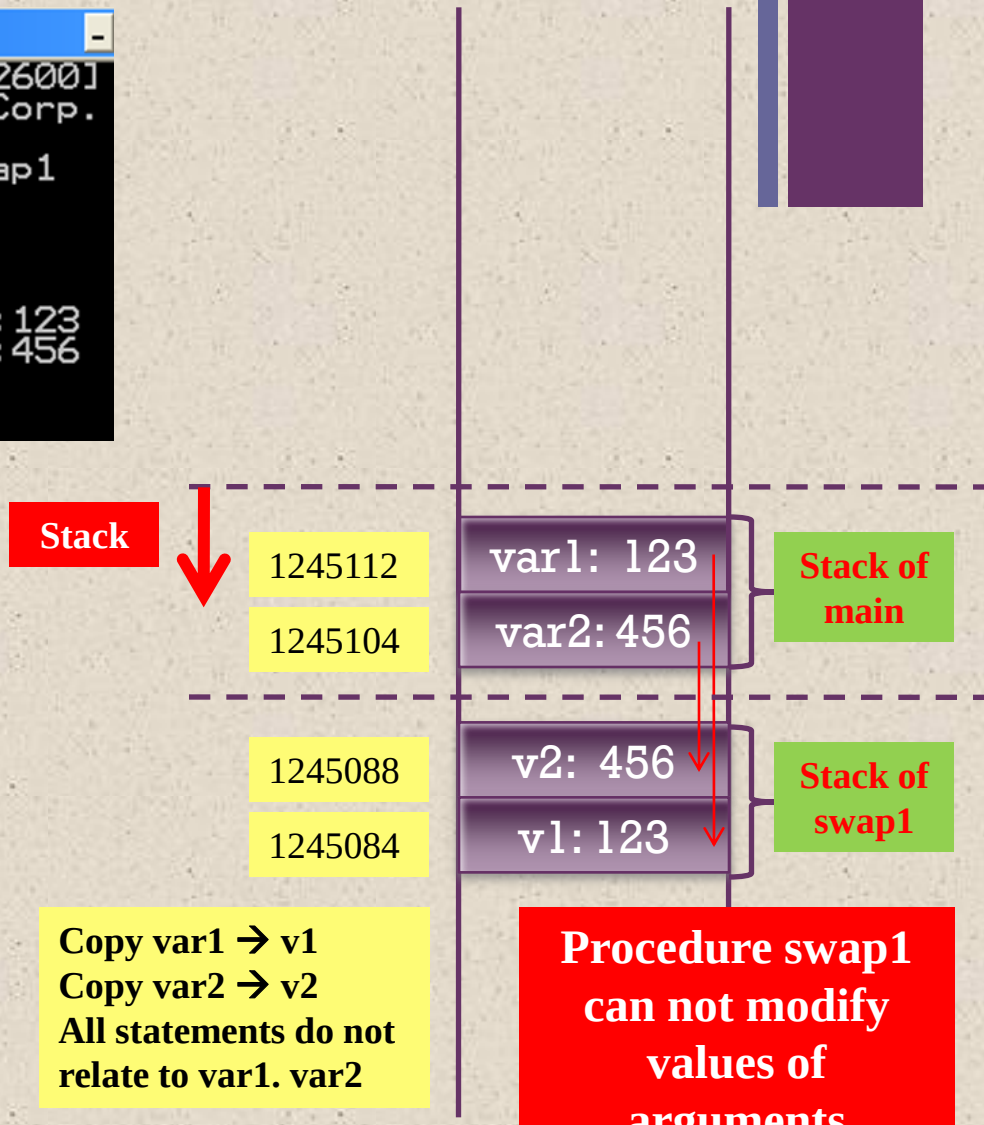
```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

K:\GiangDay\FU\CEA\ASM-Examples>swap1
Enter number 1 : 123
Enter number 2 : 456
var1, address:1245112, value:123
var2, address:1245104, value:456
Argument 1, address:1245084, value:123
Argument 2, address:1245088, value:456
After swapping:123, 456
K:\GiangDay\FU\CEA\ASM-Examples>_
```

From **main**

```
invoke swap1, var1 , var2
```

call

```
swap1 proc v1: DWORD, v2:DWORD
  ; Print out information of arguments
    print chr$("Argument 1, address:")
    lea eax, v1
    print str$(eax)
    print chr$(", value:")
    print str$(v1)
    print chr$(13,10)
    print chr$("Argument 2, address:")
    lea eax, v2
    print str$(eax)
    print chr$(", value:")
    print str$(v2)
    print chr$(13,10)
  ; swap values
    mov eax, v1 ; eax= v1
    mov ebx, v2 ; ebx= v1
    mov v1, ebx
    mov v2, eax
    ret
swap1 endp
```



Copy var1 → v1
Copy var2 → v2
All statements do not relate to var1. var2

Procedure swap1 can not modify values of arguments

Swap2.asm

- This program will repair the failure of the previous program in which a procedure is declared using pointers (address of data)

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

K:\GiangDay\FU\CEA\ASM-Examples>swap2
Enter number 1 : 123
Enter number 2 : 456
var1, address:1245112, value:123
var2, address:1245104, value:456
Argument 1, address:1245084, value:1245112
Argument 2, address:1245088, value:1245104
After swapping:456, 123
K:\GiangDay\FU\CEA\ASM-Examples>_
```

To change values of var1 and var2

t1= value at add1
t2= value at add2
Value at add1 = t2
Value at add2= t1

Swap2.asm – Source code

```
; Swap2.asm   Accept 2 integers, swap them, print results
include \masm32\include\masm32rt.inc

swap2 PROTO : PTR DWORD, : PTR DWORD ;  parameters are addresses

.code
start:
    call main
    exit
; <<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
main proc
    LOCAL var1:DWORD  ; integral variable
    LOCAL pVar1: DWORD   ; address of var1
    LOCAL var2:DWORD  ; integral variable
    LOCAL pVar2: DWORD  ; address of var2

  ; Input 2 integers
    mov var1, sval(input("Enter number 1 : "))
    mov var2, sval(input("Enter number 2 : "))

  ; Get addresses of var1, var2 to pVar1, pVar2
    lea eax, var1  ; var1, var2 are local --> use LEA
    mov pVar1, eax
    lea eax, var2
    mov pVar2, eax
```

Swap2.asm – Source code

```
; Output information of var1, var2
  print chr$("var1, address:")
  print str$(pVar1)
  print chr$(", value:")
  print str$(var1)
  print chr$(13,10)
  print chr$("var2, address:")
  print str$(pVar2)
  print chr$(", value:")
  print str$(var2)
  print chr$(13,10)

; Invoke the procedure SWAP2 to swap 2 values inputted
  push eax          ; store EAX to STACK
  push ebx          ; store EBX to STACK
  invoke swap2, pVar1 , pVar2
  pop ebx           ; restore EBX, EAX from STACK
  pop eax

; Print the result
  print chr$("After swapping:")
  print str$(var1)
  print chr$(", ")
  print str$(var2)

  ret
main endp
```

1245112 (var2)	var1: 123
1245104 (var1)	var2: 456

Stack of **main**

Swap2.asm – Source code

```
  ; Print the result
    print chr$("After swapping:")
    print str$(var1)
    print chr$(", ")
    print str$(var2)

    ret
main endp
; <<<
```

3-Loop.asm

The following program depicts a way to use loop.
Program will perform 3 times, for each time, 2 integers will be accepted then sum of them is print out.

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

K:\GiangDay\FU\CEA\ASM-Examples>loop
Program will comput sum of 2 integers 3 times
Time 1:
Enter number 1 : 12
Enter number 2 : 34
Sum of them:46
Time 2:
Enter number 1 : -90
Enter number 2 : 27
Sum of them:-63
Time 3:
Enter number 1 : 12345
Enter number 2 : 12345
Sum of them:24690

K:\GiangDay\FU\CEA\ASM-Examples>_
```

Loop.asm – Source code

```
; Loop.asm  Loop 3 times, for each time: Accept 2 integers,
; sum of them will be printed out

 include \masm32\include\masm32rt.inc

; Prototype the sum function, 2 parameters
 sum PROTO :DWORD, :DWORD

.code
start:
    call main
    exit
; <<<
```

Loop.asm – Source code

```
CONTD:                          ; Label for loop
  CMP COUNT, 0                  ; While COUNT>3 DO
  je STOP                       ; if COUNT =0, program terminates
  print chr$("Time ")
  mov eax, 4                    ; 4-3 =1, 4-2=2, 4-1=3
  SUB eax, COUNT
  print str$(eax)
  print chr$(":", 13, 10)
; Input 2 integers
  mov var1, sval(input("Enter number 1 : "))
  mov var2, sval(input("Enter number 2 : "))
; Invoke the procedure SUM to compute their sum
  invoke sum, var1 , var2
  mov result, eax   ; result = eax
; Print the result
  print chr$("Sum of them:")
  print str$(result)
  print chr$(13,10)
  DEC COUNT                 ; COUNT = COUNT -1
  JMP CONTD                 ; repeat

STOP:
  ret

main endp

; <<<
```

Exercises

Develop a program that will perform n times, for each time, 2 integers will be accepted then sum of them is printed out. Value of n is received from user.

Develop a program that will print out the nth element of the Fibonacci sequence.

Index	1	2	3	4	5	6	7	8	9	10
Fib.	1	1	2	3	5	8	13	21	34	55

```
If  (n = 1 or n =2) eax=1
Else
    t1=1; t2= 1
    loop n-2 times
          eax = t1 + t2
           t1= t2
           t2 = eax
eax contains the result
```

Exercises

Develop a procedure for computing n! using the following prototype:
factorial PROTO : DWORD

In the procedure main of the ASM program, the integer n will be accepted then value of n! will be printed out

n! = 1* 2 * 3* …. * n

Summary

- Arithmetic operations
- Accessing variable's address
- Drawing the memory map of a program
- The way procedures work.
- Using loops